

PRÁCTICA 1

PRÁCTICA 1: TÉCNICAS DE BÚSQUEDA DE POBLACIONES PARA
EL PROBLEMA DE ASIGNACIÓN CON RESTRICCIONES (PAR)



UNIVERSIDAD DE GRANADA

Metaheurísticas

Grupo A2: Viernes 15:30 - Curso 2025-2026

Jorge García Gámiz

DNI: 77449859G

e-mail: jorgegarciag@correo.ugr.es

Índice

1. Descripción del Problema	2
2. Aplicación de los algoritmos al problema	3
3. Estructura de los algoritmos	7
3.1. Búsqueda Aleatoria	7
3.2. Algoritmo Greedy	7
3.2.1. Esquema general del algoritmo	8
3.2.2. Función heurística	9
3.3. Búsqueda Local	9
3.3.1. Operador de generación de vecino	10
3.3.2. Exploración del entorno	10
3.3.3. Generación de solución inicial	10
3.3.4. Algoritmo de Búsqueda Local	11
4. Estructura del código	13
4.1. Manual de compilación	13
4.2. Manual de ejecución	14
5. Experimentación y Análisis de Resultados	15
5.1. Descripción de los casos de prueba	15
5.2. Parámetros de ejecución	15
5.3. Resultados obtenidos	16
5.4. Análisis de resultados	17
5.4.1. Comparación general entre algoritmos	17
5.4.2. Impacto de las restricciones en la función objetivo	18
5.4.3. Esfuerzo computacional y convergencia	18
5.4.4. Comparación entre conjuntos de datos y dimensionalidad	19
5.4.5. Análisis de la variabilidad de los resultados	19
5.4.6. Conclusiones del análisis	22
5.5. Contraste de Hipótesis mediante el Test de Wilcoxon	23
6. Referencias y recursos utilizados	25

1. Descripción del Problema

El Problema de Asignación con Restricciones (PAR) es una generalización del problema de agrupamiento clásico que busca particionar un conjunto de n instancias en k clusters disjuntos, donde cada instancia $\vec{x}_i \in \mathbb{R}^d$ tiene d características numéricas.

Además, se dispone de una matriz de restricciones $R \in M_{n \times n}$, donde:

- $R_{ij} = 1$ es Must-Link (ML): i y j deben pertenecer al mismo cluster
- $R_{ij} = -1$ es Cannot-Link (CL): i y j deben estar en distintos clusters
- $R_{ij} = 0$ indica ausencia de restricción.

Una solución se representa como un vector de asignación $s = (s_1, \dots, s_n)$ con $s_i \in \{0, \dots, k-1\}$ asigna cada instancia a un cluster.

Función Objetivo

El objetivo es obtener una partición solución que permita minimizar la función fitness definida como:

$$fitness(s) = desviacion(s) + \lambda \cdot infeasibility(s), \quad (1)$$

donde el parámetro $\lambda = \frac{\max_{i,j} \|\vec{x}_i - \vec{x}_j\|}{R} \in [0, 1[$ equilibra ambos términos (R es el número de restricciones).

Desviación

La calidad del clustering se mide mediante la desviación general. Primero se calculan los centroides. Para cada cluster C_i , con $i \in \{0, \dots, k-1\}$, su centroide es:

$$\vec{\mu}_i = \frac{1}{|C_i|} \sum_{\vec{x}_j \in C_i} \vec{x}_j$$

Luego la distancia media intra-cluster, que es la media de las distancias de las instancias que lo conforman a su centroide:

$$dist_{intra-cluster}(C_i) = \frac{1}{|C_i|} \sum_{\vec{x}_j \in C_i} \|\vec{x}_j - \vec{\mu}_i\|$$

Definimos la desviación general de la partición en clusters $C = \{C_1, \dots, C_k\}$ como la media de las desviaciones intra-cluster de cada cluster:

$$desviacion(C) = \frac{1}{k} \sum_{i=1}^k dist_{intra-cluster}(C_i)$$

Infeasibility

La “infeasibility” cuenta el número de restricciones violadas:

$$infeasibility = \sum_{(i,j) \in ML} [s_i \neq s_j] + \sum_{(i,j) \in CL} [s_i = s_j]$$

2. Aplicación de los algoritmos al problema

Para la resolución del problema de Asignación con Restricciones (PAR), hemos implementado tres algoritmos distintos: Búsqueda Aleatoria (Random Search), Búsqueda Greedy y Búsqueda Local. Todos ellos comparten una estructura común, heredada de la clase MH, que únicamente incluye un constructor y un destructor por defecto y el método `optimize`, el cual recibe como parámetros el problema que van a estudiar y el número máximo de evaluaciones antes de parar la ejecución del algoritmo. Este es el método que se llamará desde el main para estudiar cómo se comporta cada algoritmo a la hora de resolver el problema.

Los tres algoritmos implementados comparten una misma estructura básica de representación de soluciones y evaluación de la función objetivo. En esta sección se describen los elementos comunes a todos los métodos: la representación de soluciones, la generación de soluciones iniciales y el cálculo del fitness.

Representación del problema

El problema se encapsula en la clase `ProblemPAR`, encargada de almacenar los datos de la instancia y proporcionar los métodos necesarios para evaluar soluciones. Esta clase mantiene como atributos principales:

- `features`: matriz que contiene las características de cada instancia del conjunto de datos.
- `constraints`: matriz de restricciones que indica las relaciones Must-Link y Cannot-Link entre pares de instancias.
- `num_clusters`: número de clusters en los que se debe dividir el conjunto de instancias.
- `lambda`: parámetro de penalización utilizado en la función objetivo.

Durante la construcción del problema se leen los datos desde fichero y se calcula automáticamente el valor de λ según la expresión definida en la sección anterior.

Representación de las soluciones

Una solución candidata se representa mediante una estructura `tSolution<int>`, que, como ya describimos en la sección anterior, en nuestra implementación corresponde a un vector de enteros donde cada posición i indica el identificador del cluster al que pertenece la instancia i . Por tanto:

$$s = (s_1, s_2, \dots, s_n), \quad s_i \in \{0, \dots, k-1\}$$

donde k es el número total de clusters.

Esta representación permite modificar fácilmente la asignación de cualquier instancia a otro cluster, lo cual es fundamental para los algoritmos de búsqueda utilizados.

Generación de soluciones iniciales

Los algoritmos que requieren una solución inicial utilizan el método `createSolution`, que genera una asignación aleatoria de instancias a clusters. El procedimiento consiste simplemente en asignar a cada instancia un cluster elegido de forma uniforme dentro del dominio permitido.

El pseudocódigo del procedimiento es el siguiente:

```
createSolution()
n <- número de instancias
solucion <- vector de tamaño n

para i desde 0 hasta n-1 hacer
    solucion[i] <- valor aleatorio en [0, k-1]

devolver solucion
```

Este método permite generar rápidamente soluciones iniciales diversas que pueden ser utilizadas por los distintos algoritmos de búsqueda.

Cálculo de centroides

Para evaluar la calidad de una solución es necesario calcular los centroides de los clusters definidos por dicha solución. El procedimiento utilizado es el siguiente:

```
calculateCentroids(solution)

inicializar centroides[k][d] a 0
inicializar contador_elementos[k] a 0 para cada cluster

para cada instancia i hacer
    c <- solution[i]
    centroides[c] <- centroides[c] + features[i]
    contador_elementos[c]++

para cada cluster c hacer
    centroides[c] <- centroides[c] / contador_elementos[c]

devolver centroides
```

donde d representa el número de características de cada instancia.

Cálculo de la desviación

Una vez obtenidos los centroides, se calcula la desviación media intra-cluster de cada cluster y posteriormente la desviación global de la partición.

```
calculateDeviation(solution)

centroides <- calculateCentroids(solution)

para cada cluster c hacer
  suma_distancias[c] <- 0
  contador[c] <- 0

para cada instancia i hacer
  si solution[i] = c entonces
    suma_distancias += distancia(x_i , centroide_c)
    contador[c]++
para cada cluster c
  si contador[c] > 0 entonces
    desviacion_cluster[c] <- suma_distancias[c] / contador[c]
  sino
    desviacion_cluster[c] = PENALTY

devolver media(desviacion_cluster)
```

Si algún cluster queda vacío, se aplica una penalización elevada (999999.0) para evitar que este tipo de soluciones sea favorecido por los algoritmos de búsqueda.

Cálculo de infeasibility

El término de infeasibility mide el número de restricciones violadas por una solución. Para ello se recorren todos los pares de instancias y se comprueba si la asignación de clusters respeta las restricciones Must-Link y Cannot-Link.

```
calculateInfeasibility(solution)

infeasibility <- 0

para cada par de instancias (i,j) con i < j hacer
  si constraint[i][j] = 1 y solution[i] != solution[j] entonces
    infeasibility++
  si constraint[i][j] = -1 y solution[i] = solution[j] entonces
    infeasibility++

devolver infeasibility
```

Función objetivo

Finalmente, el valor de fitness de una solución se calcula (véase Ecuación (1)) combinando la desviación intra-cluster y el número de restricciones violadas mediante el parámetro λ descrito anteriormente. Así se refleja en el código:

```
fitness(solution)

desviacion <- calculateDeviation(solution)
infeas <- calculateInfeasibility(solution)

devolver desviacion + lambda * infeas
```

Esta función permite equilibrar la calidad del agrupamiento con el grado de cumplimiento de las restricciones, penalizando aquellas soluciones que violan un mayor número de ellas.

Los tres algoritmos implementados utilizan esta misma función de evaluación para comparar soluciones y guiar el proceso de búsqueda.

3. Estructura de los algoritmos

En esta sección se describen los algoritmos utilizados para resolver el problema. En todos los casos los métodos devuelven una estructura `ResultMH`, que contiene la mejor solución encontrada, su valor de fitness y el número de evaluaciones realizadas.

3.1. Búsqueda Aleatoria

El algoritmo Random Search constituye un método de referencia muy sencillo que consiste en generar soluciones completamente aleatorias y conservar la mejor encontrada.

En cada iteración se genera una solución mediante el método `createSolution` definido en el problema y se evalúa mediante la función `fitness`. Si la nueva solución mejora a la mejor encontrada hasta el momento, se actualiza.

El proceso continúa hasta alcanzar el número máximo de evaluaciones permitido.

```
RandomSearch(problem, maxevals)

best_solution <- null
best_fitness <- infinito
evaluations <- 0

para i desde 1 hasta maxevals hacer
    solution <- createSolution(problem)
    fitness <- fitness(solution)

    evaluations++

    si fitness < best_fitness entonces
        best_solution <- solution
        best_fitness <- fitness

devolver (best_solution, best_fitness, evaluations)
```

Este algoritmo no realiza ninguna explotación del espacio de búsqueda, pero sirve como referencia para comparar el rendimiento de los métodos más sofisticados.

3.2. Algoritmo Greedy

Los algoritmos greedy se caracterizan por tomar decisiones basadas únicamente en la información disponible en el estado actual, seleccionando en cada paso la alternativa que parece más prometedora según un criterio heurístico.

En nuestro caso, el algoritmo greedy implementado sigue una estrategia similar al algoritmo *k-means*, adaptada para tener en cuenta las restricciones del problema. En lugar de optimizar directamente la función fitness global, el algoritmo construye iterativamente una partición de las instancias asignando cada una al cluster que minimiza una heurística local basada en dos criterios: el número de restricciones violadas al realizar la asignación y la distancia euclídea al centroide del cluster.

El algoritmo comienza generando k centroides iniciales aleatorios dentro del dominio de cada característica del conjunto de datos. A partir de estos centroides iniciales, las instancias se asignan iterativamente a los clusters utilizando la función heurística descrita más adelante. Tras cada iteración se recalculan los centroides como la media de las instancias asignadas a cada cluster.

El proceso se repite recalculando los centroides hasta que ninguna instancia cambia de cluster.

3.2.1. Esquema general del algoritmo

```
Greedy(problem, maxevals)
  obtener datos y parámetros (n, d, k)

  // Calcular dominio de cada característica
  calcular min_f y max_f para cada dimensión

  // Inicializar centroides aleatorios
  generar k centroides dentro del dominio

  // Inicializar solución
  sol[i] <- -1 para todo i
  changed <- true

  mientras changed hacer
    changed <- false
    barajar instancias

    para cada instancia i hacer

      mejor_cluster <- -1
      mejor_valor <- (infinito, infinito)

      para cada cluster c hacer

        valor <- Heuristica(i, c, sol, centroides, restricciones)

        si valor < mejor_valor entonces
          mejor_valor <- valor
          mejor_cluster <- c

      si sol[i] != mejor_cluster entonces
        sol[i] <- mejor_cluster
        changed <- true

  // Recalcular centroides
  para cada cluster c hacer
    centroides[c] <- media de sus instancias (si no está vacío)

  devolver sol
```

3.2.2. Función heurística

Para decidir a qué cluster asignar cada instancia se utiliza una función heurística que considera dos factores:

- Número de restricciones violadas al asignar la instancia al cluster.
- Distancia euclídea al centroide del cluster.

El criterio principal es minimizar el número de violaciones de restricciones, utilizando la distancia únicamente como criterio de desempate.

```
Heuristica(instancia i, cluster c, sol, centroides, restricciones)
```

```
// Distancia euclídea al centroide
distancia <- dist(i, centroides[c])

// Cálculo de violaciones
violaciones <- 0

para cada instancia j hacer
  si j != i y sol[j] != -1 entonces
    si restricción(i,j) = must-link y c != sol[j]
      violaciones++
    si restricción(i,j) = cannot-link y c == sol[j]
      violaciones++

devolver (violaciones, distancia)
```

Nota: la heurística no se implementa como función explícita, sino que se calcula directamente dentro del bucle de asignación.

Finalmente, cuando el algoritmo converge, la solución obtenida se evalúa mediante la función fitness definida anteriormente para obtener su valor final.

3.3. Búsqueda Local

La Búsqueda Local es un algoritmo de mejora iterativa que parte de una solución inicial y explora su entorno¹ en busca de soluciones con mejor valor de la función objetivo. El proceso continúa mientras se encuentren mejoras y no se supere el número máximo de evaluaciones permitido.

En nuestro caso el algoritmo utiliza una estrategia de exploración *primer mejor*. Esto significa que el entorno de la solución actual se recorre hasta encontrar el primer vecino que mejora el valor de la función objetivo, aceptándose inmediatamente como nueva solución. A partir de esa nueva solución se vuelve a generar y explorar su entorno.

¹formado por todas aquellas soluciones que se obtienen moviendo una instancia de su cluster actual a otro distinto

3.3.1. Operador de generación de vecino

El operador de generación de vecinos consiste en seleccionar una instancia y moverla a un cluster distinto. Formalmente, dado un vector solución $s = (s_1, \dots, s_n)$, se selecciona una instancia i y se modifica su asignación a un cluster distinto l , quedando $s' = (s_1, \dots, l, \dots, s_n)$

Para evitar generar soluciones inválidas, se impone la restricción adicional de no permitir movimientos que dejen un cluster vacío.

3.3.2. Exploración del entorno

El entorno de una solución s está formado por todas las soluciones que pueden generarse aplicando el operador anterior a cada una de las instancias y a todos los clusters distintos del actual. Por tanto, si el problema tiene n instancias y k clusters, el tamaño máximo del entorno es $n(k - 1)$.

La exploración del entorno no se realiza en un orden determinista, sino en un orden aleatorio. Esto introduce diversidad en la búsqueda y evita sesgos sistemáticos en la exploración del espacio de soluciones.

En la implementación se genera explícitamente el conjunto de movimientos posibles (i, l) y posteriormente se barajan aleatoriamente antes de comenzar la exploración.

```
generarEntorno(solucion)

    entorno <- lista vacía

    para i desde 1 hasta n hacer
        para l desde 0 hasta k-1 hacer
            si l != solucion[i]
                añadir (i,l) a entorno

    barajar(entorno)

    devolver entorno
```

La estrategia utilizada es *primer mejor*: en cuanto se encuentra un vecino con mejor fitness que la solución actual, se acepta inmediatamente y se vuelve a generar el entorno desde la nueva solución.

3.3.3. Generación de solución inicial

La búsqueda comienza generando una solución inicial aleatoria. Para evitar configuraciones inválidas se garantiza que todos los clusters contengan al menos una instancia.

El procedimiento consiste en barajar el conjunto de instancias, asignar inicialmente una instancia distinta a cada cluster y posteriormente asignar el resto de instancias a clusters aleatorios, evitando la aparición de clusters vacíos.

```
generarSolucionInicial()

  solucion <- vector tamaño n inicializado a -1
  cluster_counts <- vector tamaño k inicializado a 0

  instancias <- [0,...,n-1]
  barajar(instancias)

  // Asegurar que ningún cluster esté vacío
  para c desde 0 hasta k-1 hacer
    i <- instancias[c]
    solucion[i] <- c
    cluster_counts[c]++

  // Asignar el resto aleatoriamente
  para i desde k hasta n-1 hacer
    inst <- instancias[i]
    c <- valor aleatorio en [0, k-1]
    solucion[inst] <- c
    cluster_counts[c]++

  devolver (solucion, cluster_counts)
```

3.3.4. Algoritmo de Búsqueda Local

El algoritmo comienza evaluando la solución inicial. Posteriormente explora el entorno buscando un vecino que mejore el valor de fitness. En cuanto se encuentra una mejora, se acepta el movimiento y se vuelve a generar el entorno desde la nueva solución.

```
LocalSearch(problem, maxevals)

  (solucion, cluster_counts) <- generarSolucionInicial()

  fitness_actual <- fitness(solucion)
  evaluations <- 1
  improvement <- true

  mientras improvement y evaluations < maxevals hacer

    improvement <- false
    entorno <- generarEntorno(solucion)

    // Exploración del vecindario (Primer Mejor)
    para cada movimiento (i, new_cluster) en entorno hacer

      si evaluations >= maxevals entonces
        salir
```

```
old_cluster <- solucion[i]

// No permitir clusters vacíos
si cluster_counts[old_cluster] <= 1 entonces
    continuar

// Generar vecino
vecino <- copia(solucion)
vecino[i] <- new_cluster

fitness_vecino <- fitness(vecino)
evaluations++

// Criterio de aceptación: mejora estricta
si fitness_vecino < fitness_actual entonces

    solucion <- vecino
    fitness_actual <- fitness_vecino

// Actualizar contadores
cluster_counts[old_cluster]--
cluster_counts[new_cluster]++

improvement <- true
romper    // estrategia First Improvement

devolver solucion
```

El algoritmo finaliza cuando se alcanza el número máximo de evaluaciones de la función objetivo o cuando se explora completamente el entorno de una solución sin encontrar ninguna mejora.

4. Estructura del código

El proyecto sigue una arquitectura basada en la plantilla proporcionada en PRADO, con adaptaciones y extensiones específicas para el problema PAR y el análisis de resultados. El directorio principal `software/` se organiza de la siguiente manera:

- `common/`: Contiene las clases base y utilidades generales (`mh.h`, `problem.h`, `solution.h`, etc.). Estos archivos proporcionan la interfaz fundamental de la que heredan nuestros algoritmos y representaciones.
- `inc/` y `src/`: Directorios que separan, respectivamente, las cabeceras (`.h`) y las implementaciones (`.cpp`) de las clases desarrolladas para esta práctica. Incluyen la definición del problema (`problemPAR`) y la lógica de los tres algoritmos (`greedy`, `localsearch`, `randomsearch`) implementando el método `optimize` para cada uno.
- `bin/`: Carpeta destino donde el compilador genera los archivos objeto (`.o`) y el ejecutable final (`par_solver`).
- `data/`: Directorio que almacena los conjuntos de datos y los archivos de restricciones a evaluar.
- **Archivos en la raíz:**
 - `main.cpp`: Punto de entrada del programa que gestiona los parámetros por consola y lanza la optimización.
 - `Makefile`: Archivo de configuración para automatizar la compilación.
 - `run_all_tests.sh`: Script en Bash para la ejecución en lote de toda la batería de pruebas.
 - `graf.py`: Script en Python para la generación automática de *boxplots* a partir de los resultados.
 - `wilcoxon.py`: Script en Python que realiza el test de Wilcoxon.
 - Archivos `.csv` y directorio `graficas/`: Salidas generadas tras la ejecución de los experimentos.

4.1. Manual de compilación

Para compilar el proyecto se ha incluido un `Makefile` configurado para el compilador `g++`. Se utiliza el estándar C++17 y el flag de máxima optimización `-O3` para acelerar las ejecuciones.

Para generar el ejecutable, basta con abrir un terminal en el directorio raíz del proyecto y ejecutar el comando:

```
$ make
```

Esto compilará los archivos fuente y generará el ejecutable `par_solver` dentro de la carpeta `bin/`. Si se desea limpiar el entorno eliminando los archivos compilados, se puede ejecutar:

```
$ make clean
```

4.2. Manual de ejecución

El software ofrece flexibilidad para ejecutarse de forma individual (para pruebas concretas) o de forma automatizada (para extraer todos los resultados de la memoria).

1. Ejecución individual

El ejecutable principal admite hasta 4 parámetros por línea de comandos, siguiendo la sintaxis:

```
$ ./bin/par_solver <dataset_base_path> <k_clusters> <algorithm> [seed]
```

- `<dataset_base_path>`: Ruta base del archivo de datos (ej. `data/glass_set`). El programa buscará automáticamente su archivo de restricciones asociado.
- `<k_clusters>`: Número de clusters a formar (típicamente 15 o 30).
- `<algorithm>`: Algoritmo a ejecutar (`random`, `greedy`, `local`, o `all` para ejecutarlos todos secuencialmente).
- `[seed]` (*Opcional*): Semilla para el generador de números aleatorios. Si no se especifica, el programa realiza 5 ejecuciones independientes usando un conjunto de semillas por defecto.

Ejemplo de ejecución: Ejecutar todos los algoritmos sobre el dataset *zoo* con $k = 15$ y semilla 42:

```
$ ./bin/par_solver data/zoo_set 15 all 42
```

2. Ejecución automatizada (Batería de pruebas)

Para replicar los resultados mostrados en esta memoria, se ha desarrollado el script de bash `run_all_tests.sh`. Este script lanza internamente el programa para los datasets Bupa, Glass y Zoo (con $k = 15$ y $k = 30$) empleando las 5 semillas.

```
$ ./run_all_tests.sh
```

Al finalizar, el programa genera automáticamente los archivos `det_results.csv` (resultados detallados) y `avg_results.csv` (medias de tiempo, fitness y evaluaciones).

3. Generación de Gráficas

Una vez generados los archivos CSV, se pueden obtener los *boxplots* que comparan la estabilidad de los algoritmos ejecutando el script de Python:

```
$ python3 graf.py
```

Este comando leerá el archivo de resultados detallados y exportará las imágenes PNG dentro de la carpeta `graficas/`.

5. Experimentación y Análisis de Resultados

5.1. Descripción de los casos de prueba

Para evaluar el rendimiento y la robustez de los algoritmos implementados, se ha seleccionado un banco de pruebas compuesto por tres conjuntos de datos (*datasets*) de características variadas. Para cada uno de estos conjuntos, se ha abordado el problema de particionamiento considerando dos valores distintos para el número de clusters a formar: $k = 15$ y $k = 30$.

Las características principales de los conjuntos de datos, incluyendo el número total de instancias (n) y el número de atributos numéricos continuos que definen cada instancia (d), se resumen en la siguiente tabla:

Dataset (.dat)	Instancias (n)	Atributos (d)
bupa_set	345	5
glass_set	214	9
zoo_set	101	16

Cuadro 1: Características de los datasets empleados.

Junto a cada conjunto de datos y para cada valor de K , se proporciona un archivo de restricciones asociado (`_const_15.dat` y `_const_30.dat`), que define las condiciones estructurales *Must-Link* y *Cannot-Link* que los algoritmos deben intentar satisfacer.

5.2. Parámetros de ejecución

Dado el carácter estocástico de la Búsqueda Aleatoria y la Búsqueda Local, es necesario definir un entorno de pruebas riguroso que permita extraer conclusiones con significancia estadística. La configuración del marco experimental es la siguiente:

- **Criterio de parada:** Los algoritmos Búsqueda Aleatoria y Búsqueda Local se ejecutan utilizando como criterio de parada un límite máximo de 100 000 evaluaciones de la función objetivo. Por su parte, el algoritmo Greedy es de naturaleza constructiva y finaliza cuando el procedimiento iterativo de asignación de instancias converge, por lo que no requiere un número máximo de evaluaciones como condición de parada.
- **Inicialización del generador aleatorio:** Para mitigar el sesgo introducido por la aleatoriedad en las decisiones iniciales o en la exploración del vecindario, se han ejecutado 10 iteraciones independientes para cada combinación de algoritmo, dataset y valor de k .

Para garantizar la validez del análisis experimental, cada algoritmo se ha ejecutado múltiples veces utilizando distintas semillas aleatorias. En concreto, las ejecuciones se controlan mediante un conjunto de semillas consecutivas inicializado en el valor 1000 (es decir, $\{1000, 1001, \dots, 1009\}$), utilizadas para inicializar el generador de números aleatorios mediante la función `Random::seed()`.

De este modo, la ejecución i -ésima de cada algoritmo utiliza la misma semilla, lo que permite realizar comparaciones emparejadas entre algoritmos bajo idénticas condiciones iniciales. Esta estrategia asegura tanto la reproducibilidad de los experimentos como la validez de los contrastes estadísticos realizados posteriormente.

- **Registro de resultados:** Durante cada ejecución, el sistema ha registrado de manera automatizada el *fitness* alcanzado, la desviación, la *infeasibility* final, el número real de evaluaciones consumidas y el tiempo de cómputo en segundos.

5.3. Resultados obtenidos

En esta sección se presentan los resultados obtenidos en los experimentos realizados. Las tablas siguientes muestran los valores medios obtenidos a partir de dichas 10 ejecuciones. En concreto, se reportan las medias del valor de la función objetivo *fitness*, la desviación intra-cluster, la *infeasibility* (número de restricciones violadas), el número de evaluaciones de la función objetivo realizadas y el tiempo de ejecución en segundos.

Además de la presentación de los valores medios, se incluirán más tarde² diagramas *boxplots* que permiten visualizar la distribución de los resultados obtenidos en las 10 ejecuciones realizadas para cada algoritmo.

Algoritmo	Fitness	Distancia	Incumplimiento	Evaluaciones	Tiempo (s)
Random	0.34	0.25	490.7	100000	16.78
Greedy	0.24	0.24	17.2	1	$2,56 \times 10^{-2}$
Local	0.18	0.15	168.2	85442.6	13.54

Cuadro 2: Resultados medios para el dataset *Bupa* con $k = 15$.

Algoritmo	Fitness	Distancia	Incumplimiento	Evaluaciones	Tiempo (s)
Random	0.28	0.23	567.3	100000	29.83
Greedy	0.23	0.23	3.5	1	$3,60 \times 10^{-2}$
Local	0.13	0.10	346.7	99781.7	29.25

Cuadro 3: Resultados medios para el dataset *Bupa* con $k = 30$.

Algoritmo	Fitness	Distancia	Incumplimiento	Evaluaciones	Tiempo (s)
Random	0.52	0.40	173.5	100000	6.32
Greedy	0.34	0.34	0	1	$1,01 \times 10^{-2}$
Local	0.22	0.19	47.2	36397	2.07

Cuadro 4: Resultados medios para el dataset *Glass* con $k = 15$.

²en la sección 5.4.5: Análisis de la variabilidad de los resultados

Algoritmo	Fitness	Distancia	Incumplimiento	Evaluaciones	Tiempo (s)
Random	0.42	0.35	218.7	100000	11.20
Greedy	73333.5	73333.5	0	1	$1,80 \times 10^{-2}$
Local	0.16	0.13	103.6	76518.1	8.15

Cuadro 5: Resultados medios para el dataset *Glass* con $k = 30$.

Algoritmo	Fitness	Distancia	Incumplimiento	Evaluaciones	Tiempo (s)
Random	1.47	1.27	37	100000	1.30
Greedy	13334.2	13334.2	0	1	$1,64 \times 10^{-3}$
Local	0.53	0.50	6.3	13951.9	0.12

Cuadro 6: Resultados medios para el dataset *Zoo* con $k = 15$.

Algoritmo	Fitness	Distancia	Incumplimiento	Evaluaciones	Tiempo (s)
Random	1.03	0.90	48.7	100000	1.99
Greedy	140000	140000	0	1	$2,93 \times 10^{-3}$
Local	0.29	0.24	17.5	23325.4	0.24

Cuadro 7: Resultados medios para el dataset *Zoo* con $k = 30$.

5.4. Análisis de resultados

Una vez presentados los resultados promedios de las ejecuciones, procedemos a realizar un análisis exhaustivo del comportamiento de cada algoritmo. El objetivo es interpretar los resultados en función de las características de cada algoritmo y de la función objetivo empleada.

5.4.1. Comparación general entre algoritmos

De forma general, el algoritmo de *Búsqueda Local* obtiene los mejores valores de *fitness* en todos los conjuntos de datos y para ambos valores de k . Este comportamiento es esperable, ya que este método parte de una solución inicial aleatoria y explora iterativamente el vecindario aceptando mejoras, lo que le permite refinar progresivamente la calidad de la solución.

Por el contrario, el algoritmo de *Búsqueda Aleatoria* presenta los peores resultados en la mayoría de los casos. Este comportamiento también resulta coherente con su funcionamiento, ya que el algoritmo simplemente genera soluciones aleatorias dentro del espacio de búsqueda sin realizar ningún proceso de mejora posterior. Como consecuencia, aunque puede encontrar ocasionalmente soluciones aceptables, en promedio la calidad de las soluciones generadas es significativamente inferior.

El algoritmo *Greedy* muestra un comportamiento intermedio en algunos casos, aunque presenta resultados muy variables dependiendo del conjunto de datos. Al tratarse de un algoritmo constructivo que toma decisiones locales basadas en la información disponible en cada paso, puede generar soluciones razonables cuando la estructura del problema lo favorece, pero también puede quedar atrapado en configuraciones poco adecuadas.

5.4.2. Impacto de las restricciones en la función objetivo

La formulación de la función objetivo (1) condiciona de forma directa el comportamiento de los algoritmos frente al cumplimiento de las restricciones. El equilibrio entre los términos de desviación intra-cluster e *infeasibility* determina el tipo de soluciones que cada algoritmo tiende a generar.

En este contexto, el algoritmo *Greedy* muestra una clara tendencia a priorizar la satisfacción de las restricciones desde las primeras fases de construcción de la solución. Como resultado, en muchas ejecuciones se obtienen particiones con *infeasibility* igual a cero. Sin embargo, esta estrategia puede deteriorar significativamente la calidad geométrica de los clusters, incrementando la desviación intra-cluster.

Este comportamiento se hace especialmente evidente en algunos casos donde se observan valores extremadamente elevados del *fitness* para el algoritmo *Greedy*, particularmente en los datasets *Glass* (superando los 73333), como se muestra en el Cuadro 5, y *Zoo* (superando los 140000), como puede observarse en el Cuadro 7.

Estos valores resultan poco coherentes desde el punto de vista del clustering, ya que implican desviaciones intra-cluster extremadamente elevadas. Una posible explicación de este fenómeno está relacionada con la naturaleza constructiva del algoritmo. Al asignar instancias de forma secuencial sin mecanismos de retroceso (*backtracking*), el método puede generar configuraciones problemáticas cuando el número de clusters es elevado en relación con el tamaño del dataset. En conjuntos de datos relativamente pequeños (como *Zoo*, con $n = 101$ instancias) la imposición de $k = 30$ clusters puede conducir a situaciones en las que algunos clusters quedan vacíos. La función objetivo penaliza explícitamente estas configuraciones mediante una penalización muy elevada, lo que provoca que el valor final del *fitness* se dispare.

A pesar de este comportamiento en ciertos escenarios, el algoritmo *Greedy* presenta la ventaja de generar desde el inicio soluciones completamente factibles en términos de restricciones. En contraste, la *Búsqueda Local* tiende a priorizar la mejora de la estructura geométrica de los clusters, reduciendo la distancia de las instancias a sus centroides incluso cuando ello implica violar algunas restricciones. Esta diferencia de comportamiento refleja el compromiso inherente entre calidad geométrica y viabilidad relacional que introduce la función objetivo (1).

5.4.3. Esfuerzo computacional y convergencia

El análisis de las evaluaciones consumidas revela dinámicas de convergencia muy distintas. El algoritmo de Búsqueda Aleatoria, por diseño, agota sistemáticamente el presupuesto máximo de 100000 evaluaciones. El algoritmo *Greedy*, por su parte, realiza una única evaluación constructiva, siendo extremadamente rápido (esto se refleja en cualquiera de las tablas, por ejemplo en Cuadro 3, comparado con los otros dos algoritmos).

El comportamiento más destacable reside en la *Búsqueda Local* y su convergencia. Aunque dispone del mismo límite de 100000 evaluaciones que el aleatorio, casi nunca lo agota (si bien se queda cerca en el Cuadro 3). En el conjunto *zoo_set* ($k = 30$) el algoritmo converge en solamente 14000 evaluaciones, como muestra el Cuadro 6. Esta rápida detención sugiere que el espacio de búsqueda en este dataset cuenta con óptimos locales muy profundos o vecindarios altamente restrictivos que impiden

seguir explorando rápidamente, ahorrando tiempo de cómputo.

5.4.4. Comparación entre conjuntos de datos y dimensionalidad

El comportamiento de los algoritmos también varía en función del conjunto de datos considerado. En el caso de `bupa_set` y `glass_set`, la diferencia entre los algoritmos es más pronunciada, observándose una mejora clara al pasar de Búsqueda Aleatoria a Búsqueda Local.

Al evaluar el impacto de la dimensionalidad ($k = 15$ frente a $k = 30$), observamos dos consecuencias a resañar. Por un lado, aumentar el número de clusters mejora (reduce) la desviación intra-cluster en la Búsqueda Local; geoméricamente, disponer de más centroides facilita que cualquier instancia esté cerca de uno de ellos.

Por otro lado, el incremento del número de clusters tiende a aumentar la dificultad del problema. Esto se observa especialmente en el incremento del número de restricciones violadas, lo que repercute directamente en el valor final de la función objetivo. En el `zoo_set`, la Búsqueda Local pasa de tener un infeasibility de 5,62 con $k = 15$ a acumular 17,28 restricciones violadas con $k = 30$, como reflejan los Cuadros 6 y 7.

5.4.5. Análisis de la variabilidad de los resultados

Como ya hemos comentado, con el objetivo de analizar la estabilidad de los algoritmos evaluados, cada uno de ellos se ejecutó 10 veces para cada combinación de dataset y número de clusters. A partir de estas ejecuciones se construyeron los *box-plots*, que permiten visualizar la distribución de los valores de *fitness*, así como su dispersión, rango intercuartílico y la posible presencia de valores atípicos (*outliers*).

Dataset `bupa_set` En la Figura 1 se muestra la distribución del *fitness* para los tres algoritmos en el dataset `bupa_set` para ambos valores de k .

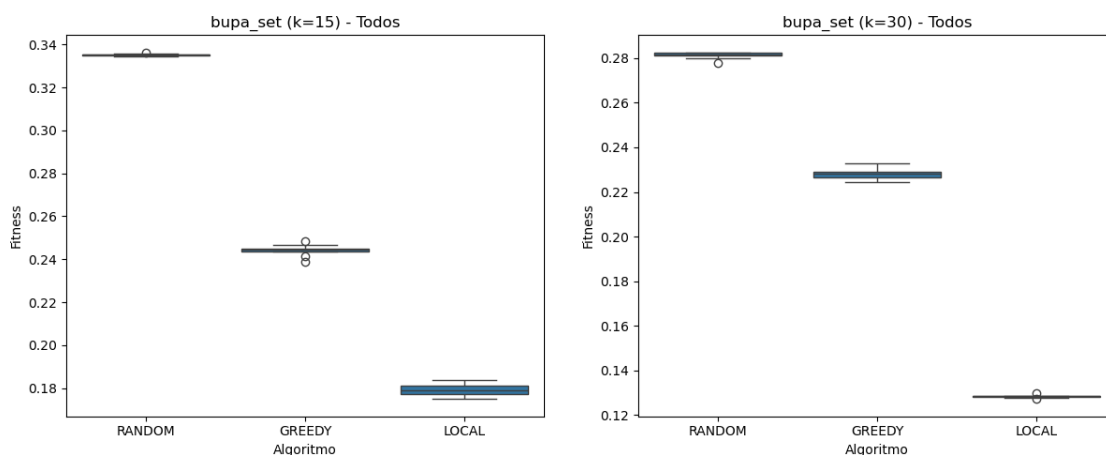


Figura 1: Distribución del fitness para el dataset *Bupa* con $k = 15$ (izq) y $k = 30$ (dcha)

En ambos casos se observa una dispersión reducida en los resultados, lo que indica un comportamiento estable entre ejecuciones. El algoritmo *Random* presenta

valores de *fitness* consistentemente más altos, mientras que *Greedy* mejora estos resultados de forma sistemática. Por su parte, la *Búsqueda Local* no solo alcanza los mejores valores, sino que además muestra una variabilidad muy baja, lo que indica una convergencia robusta hacia soluciones de calidad independientemente de la inicialización.

Dataset `glass_set` La Figura 2 muestra la distribución de resultados para el dataset `glass_set`.

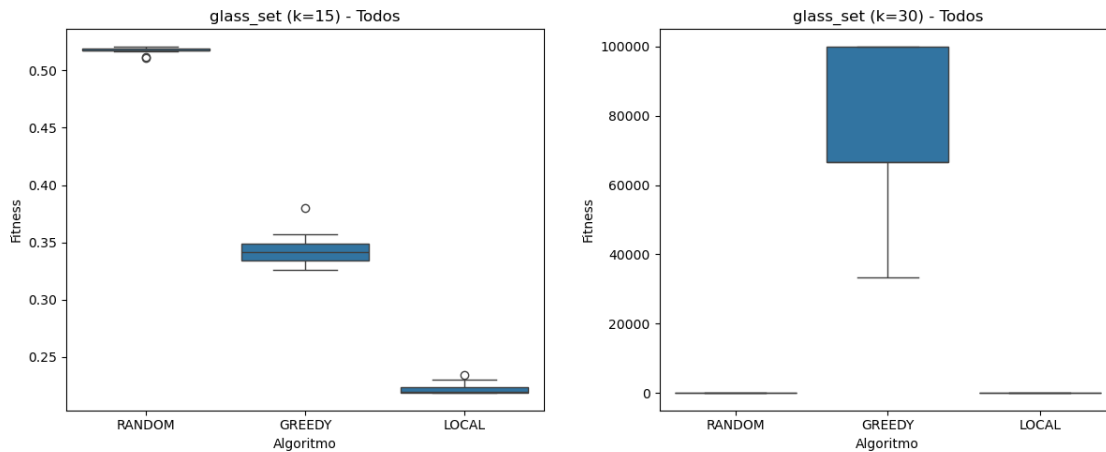


Figura 2: Distribución del fitness para el dataset *Glass* con $k = 15$ (izq) y $k = 30$ (dcha)

Para $k = 15$ se observa un comportamiento similar al descrito anteriormente: las distribuciones son relativamente compactas y existe una clara mejora al pasar de *Búsqueda Aleatoria* a *Búsqueda Local*.

Sin embargo, para $k = 30$ el comportamiento del algoritmo *Greedy* cambia drásticamente. En este caso aparecen valores extremadamente elevados de *fitness*, ya analizados en la subsección anterior, que están asociados a penalizaciones debidas a configuraciones estructuralmente problemáticas (clusters vacíos). Estos valores distorsionan la escala del eje vertical e impiden observar con claridad la distribución de los resultados de los otros algoritmos.

Para facilitar el análisis de la variabilidad entre *Búsqueda Aleatoria* y *Búsqueda Local*, se incluye en la Figura 3 una representación ampliada donde únicamente se comparan estos dos métodos.

En esta representación se observa que *Búsqueda Local* mejora de forma clara y consistente a *Búsqueda Aleatoria*, sin solapamiento entre distribuciones. Además, ambos algoritmos presentan baja dispersión, lo que indica estabilidad, aunque con diferencias muy significativas en la calidad de las soluciones.

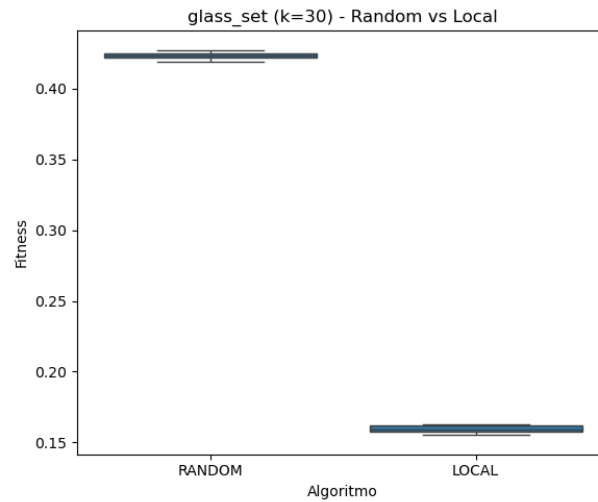


Figura 3: Comparación ampliada entre Búsqueda Aleatoria y Búsqueda Local para *glass_set* con $k = 30$

Dataset *zoo_set* Finalmente, la Figura 4 muestra los resultados correspondientes al dataset *zoo_set*.

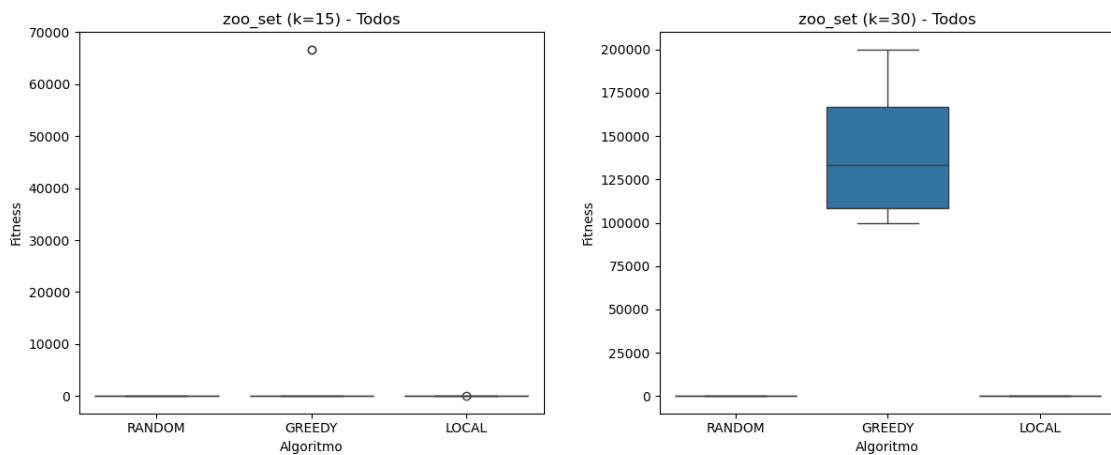


Figura 4: Distribución del fitness para el dataset *Zoo* con $k = 15$ (izq) y $k = 30$ (dcha)

En este dataset se observa nuevamente la aparición de valores extremos en el algoritmo *Greedy*. Para $k = 15$, este comportamiento se manifiesta en un único outlier muy elevado, mientras que para $k = 30$ la dispersión es mucho mayor y afecta a un número significativo de ejecuciones. En ambos casos, estos valores están asociados a penalizaciones por configuraciones inválidas, lo que evidencia una falta de robustez del método constructivo.

Para analizar adecuadamente el comportamiento del resto de algoritmos, se presentan en la Figura 5 las correspondientes versiones ampliadas.

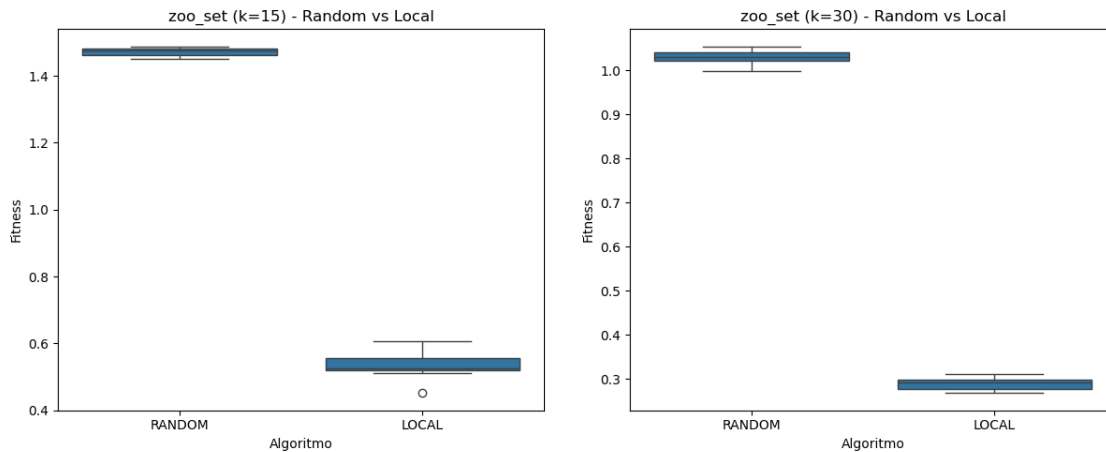


Figura 5: Comparación ampliada entre Búsqueda Aleatoria y Búsqueda Local para *zoo_set* con $k = 15$ (izq) y $k = 30$ (dcha)

Estas gráficas permiten observar con claridad que la *Búsqueda Local* obtiene valores de *fitness* considerablemente menores que la *Búsqueda Aleatoria* en ambas configuraciones. Aunque se aprecia una ligera dispersión en los resultados del algoritmo de mejora local, todos ellos se mantienen dentro de rangos claramente superiores en calidad respecto a los obtenidos mediante soluciones completamente aleatorias.

En conjunto, el análisis de los *boxplots* confirma que las mejoras observadas en los valores medios de *fitness* no dependen de ejecuciones aisladas, sino que se mantienen de forma consistente a lo largo de las distintas repeticiones experimentales. Esto refuerza la conclusión de que la *Búsqueda Local* no solo produce mejores soluciones en promedio, sino que además presenta un comportamiento robusto frente a la variabilidad inherente a la inicialización aleatoria, mientras que *Greedy* presenta una alta sensibilidad a la estructura del problema, lo que puede derivar en soluciones muy degradadas en determinados escenarios.

5.4.6. Conclusiones del análisis

A partir de los resultados obtenidos, se puede concluir que la *Búsqueda Local* es el método más eficaz entre los algoritmos evaluados para el problema de clustering con restricciones considerado. Su capacidad para mejorar iterativamente soluciones iniciales le permite encontrar configuraciones de clusters mayor calidad, logrando un mejor equilibrio entre la desviación intra-cluster y el cumplimiento de las restricciones.

Por su parte, la *Búsqueda Aleatoria* actúa como una referencia base, mostrando un comportamiento estable pero limitado, debido a la ausencia de mecanismos de explotación del espacio de soluciones.

Finalmente, el algoritmo *Greedy* presenta un comportamiento dependiente del contexto. Aunque puede generar soluciones factibles de forma muy eficiente, especialmente en escenarios donde el tamaño del dataset es suficientemente grande en relación con el número de clusters ($n \gg k$), pero su naturaleza constructiva lo hace más susceptible a generar configuraciones poco adecuadas en determinados conjuntos de datos.

5.5. Contraste de Hipótesis mediante el Test de Wilcoxon

A lo largo del análisis previo, la inspección visual de los *boxplots* y los valores promedio ha sugerido una clara superioridad del algoritmo de *Búsqueda Local* frente a la *Búsqueda Aleatoria*. No obstante, con el fin de dotar a esta comparación de mayor rigor estadístico, se ha realizado un contraste de hipótesis.

Dado que ambos algoritmos han sido evaluados bajo condiciones emparejadas (utilizando las mismas 10 semillas aleatorias en cada experimento) y que no puede asumirse *a priori* la normalidad de las distribuciones del *fitness*, no resulta apropiado aplicar un test paramétrico como el test de Student. En su lugar, se ha empleado el **test de los rangos signados de Wilcoxon**, que es un test no paramétrico.

Las hipótesis consideradas son:

- **Hipótesis nula (H_0):** No existen diferencias significativas entre las medianas del *fitness* obtenido por la Búsqueda Aleatoria y la Búsqueda Local.
- **Hipótesis alternativa (H_1):** La Búsqueda Local obtiene valores de *fitness* significativamente menores que la Búsqueda Aleatoria.

Para la realización del contraste se ha desarrollado un *script* en Python utilizando la librería `scipy.stats`. El test se ha aplicado de forma independiente para cada conjunto de datos (*Bupa*, *Glass*, *Zoo*) y para cada valor de k (15 y 30), fijando un nivel de significación $\alpha = 0,05$. Los resultados obtenidos son los siguientes:

RESULTADOS DEL TEST DE WILCOXON (RandomSearch vs LocalSearch)

```
-----
Dataset: data/bupa_set    | k: 15 | p-valor: 9.766e-04
Diferencia significativa (LocalSearch es estadísticamente mejor)

Dataset: data/bupa_set    | k: 30 | p-valor: 9.766e-04
Diferencia significativa (LocalSearch es estadísticamente mejor)

Dataset: data/glass_set   | k: 15 | p-valor: 9.766e-04
Diferencia significativa (LocalSearch es estadísticamente mejor)

Dataset: data/glass_set   | k: 30 | p-valor: 9.766e-04
Diferencia significativa (LocalSearch es estadísticamente mejor)

Dataset: data/zoo_set     | k: 15 | p-valor: 9.766e-04
Diferencia significativa (LocalSearch es estadísticamente mejor)

Dataset: data/zoo_set     | k: 30 | p-valor: 9.766e-04
Diferencia significativa (LocalSearch es estadísticamente mejor)
```

Interpretación y conclusiones del contraste

En todos los escenarios analizados, los *p-valores* obtenidos mediante el test de Wilcoxon son claramente inferiores al nivel de significación establecido ($p \approx 9,77 \times 10^{-4} <$

0,05). Por tanto, los datos aportan evidencia para rechazar la hipótesis nula (H_0) en favor de la hipótesis alternativa (H_1).

Un aspecto relevante es que el mismo *p-valor* aparece en todos los casos. Esto no es casual, sino consecuencia del tamaño de la muestra ($N = 10$ pares) y del hecho de que el algoritmo de *Búsqueda Local* obtiene sistemáticamente mejores resultados que la *Búsqueda Aleatoria* en todas las comparaciones emparejadas. En estas condiciones, el test de Wilcoxon alcanza su valor mínimo posible, que viene dado por $1/2^{10} = 1/1024 \approx 9,77 \times 10^{-4}$.

Este resultado indica que la probabilidad de observar una mejora tan consistente bajo la hipótesis nula es extremadamente baja. En consecuencia, puede concluirse que la superioridad de la *Búsqueda Local* frente a la *Búsqueda Aleatoria* no solo es apreciable desde el punto de vista empírico, sino que también resulta estadísticamente significativa en todos los escenarios analizados.

6. Referencias y recursos utilizados

Para la realización de esta práctica y la elaboración de la presente memoria, se han consultado y utilizado los siguientes recursos:

- **Material de la asignatura:** Diapositivas, guión de prácticas y recursos de PRADO.
- **Herramientas de Inteligencia Artificial:** Se ha hecho uso de herramientas de IA generativa como apoyo en tareas de programación y redacción. En concreto, se han empleado para:
 - Resolver dudas puntuales de sintaxis en C++ y mejorar la organización del archivo `Makefile`.
 - Desarrollar el script de Python (`graf.py`), utilizando las librerías `pandas`, `matplotlib` y `seaborn`, para la generación automatizada de gráficos (*boxplots*) y el procesamiento de los datos almacenados en archivos `.csv`.
 - Asistir en el formateo, revisión y estructuración del documento final en $\text{\LaTeX}$.
- Documentación oficial de C++ (cppreference.com) para el manejo de estructuras de datos estándar, y documentación oficial de las librerías de Python empleadas para la visualización de datos.
- Para la realización del análisis estadístico de los resultados, en la aplicación del test de hipótesis de Wilcoxon, se ha utilizado el material de la asignatura *Inferencia Estadística*, dentro de la parte de Matemáticas del doble grado.